

Database Transactions

1. Definition: Database Transactions and Related Terms

Database Transaction

A **database transaction** is a **logical unit of work** that contains one or more SQL statements executed as a single unit. A transaction must be either **fully completed** or **fully failed**—there is no partial execution.

Key Related Terms

- **ACID Properties:** Set of properties that ensure reliable processing of database transactions:
 - **Atomicity:** All operations in a transaction are completed; if one fails, all fail.
 - **Consistency:** Transactions take the database from one valid state to another.
 - **Isolation:** Concurrent transactions do not interfere with each other.
 - **Durability:** Once a transaction is committed, its changes are permanent.
- **Commit:** Finalizes a transaction, making all changes permanent.
- **Rollback:** Undoes changes made by a transaction before it is committed.
- **Savepoint:** A point within a transaction that allows partial rollback.
- **Concurrency:** Multiple transactions occurring simultaneously without conflict.
- **Deadlock:** A situation where two or more transactions are waiting indefinitely for one another to release locks.

2. Database Transaction Process (Outline)

1. **Begin Transaction**
 - The database starts tracking operations.
2. **Execute Operations**
 - SQL commands (INSERT, UPDATE, DELETE, etc.) are executed.
3. **Check for Errors**
 - Ensure operations do not violate constraints or cause failure.
4. **Commit or Rollback**
 - If all operations succeed → **Commit**.
 - If any operation fails → **Rollback** to undo all changes.
5. **End Transaction**
 - System is ready for the next transaction.

Flow Diagram:

Start → Begin Transaction → Execute SQLs

↓

↓

Check Errors → Yes → Rollback → End



No Errors → Commit → End

3. ACID Properties in Practice

- **Atomicity Example:** Transferring money from Account A to Account B should either deduct from A and add to B—or not happen at all.
- **Consistency Example:** After a transfer, the total amount across both accounts should remain constant.
- **Isolation Example:** Two customers placing an order at the same time should not interfere with each other's inventory records.
- **Durability Example:** Even after a power failure, committed changes remain intact in the database.

4. Writing an SQL Transaction

Here is an example of a SQL transaction that transfers money between two bank accounts:

```
BEGIN TRANSACTION;

UPDATE accounts
SET balance = balance - 100
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE account_id = 2;

-- Check if both updates are successful

-- (In practice, you'd include error handling logic)

COMMIT;
```

Rollback Example (on error):

```
BEGIN TRANSACTION;

UPDATE accounts
SET balance = balance - 100
```

```
WHERE account_id = 1;
```

```
-- Suppose this fails:
```

```
UPDATE accounts
```

```
SET balance = balance + 100
```

```
WHERE account_id = 999; -- Nonexistent account
```

```
ROLLBACK;
```